Search models, datasets, users...

← Back to Articles

Transformers.js v3: WebGPU Support, New Models & Tasks, and More…

Published October 22, 2024

Update on GitHub

Upvote 77 +71

Joshua
Xenova Follow

After more than a year of development, we're excited to announce the release of 🤗 Transformers.js v3!

Highlights include:

- WebGPU support (up to 100x faster than WASM!)
- New quantization formats (dtypes)
- A total of 120 supported architectures
- 25 new example projects and templates
- Over 1200 pre-converted models on the Hugging Face Hub
- Node.js (ESM + CJS), Deno, and Bun compatibility
- A new home on GitHub and NPM

Installation

You can get started by installing Transformers.js v3 from NPM using:

```
npm i @huggingface/transformers
```

Then, importing the library with

```
import { pipeline } from "@huggingface/transformers";
```

or, via a CDN

```
import { pipeline } from "https://cdn.jsdelivr.net/npm/@huggingface/tr
```

For more information, check out the documentation.

WebGPU support

WebGPU is a new web standard for accelerated graphics and compute. The API enables web developers to use the underlying system's GPU to carry out high-performance computations directly in the browser. WebGPU is the successor to WebGL and provides significantly better performance, because it allows for more direct interaction with modern GPUs. Lastly, it supports general-purpose GPU computations, which makes it just perfect for machine learning!

> As of October 2024, global WebGPU support is around 70% (according to caniuse.com), meaning some users may not be able to use the API.
>
> If the following demos do not work in your browser, you may need to enable it using a feature flag:
>
> - Firefox: with the `dom.webgpu.enabled` flag (see here).
> - Safari: with the `WebGPU` feature flag (see here).
> - Older Chromium browsers (on Windows, macOS, Linux): with the `enable-unsafe-webgpu` flag (see here).

Usage in Transformers.js v3

Thanks to our collaboration with ONNX Runtime Web, enabling WebGPU acceleration is as simple as setting `device: 'webgpu'` when loading a model. Let's see some examples!

Example: Compute text embeddings on WebGPU (demo)

```
import { pipeline } from "@huggingface/transformers";

// Create a feature-extraction pipeline
const extractor = await pipeline(
  "feature-extraction",
  "mixedbread-ai/mxbai-embed-xsmall-v1",
  { device: "webgpu" },
);

// Compute embeddings
const texts = ["Hello world!", "This is an example sentence."];
const embeddings = await extractor(texts, { pooling: "mean", normalize
console.log(embeddings.tolist());
// [
//   [-0.016986183822155, 0.03228696808218956, -0.0013630966423079371,
//   [0.09050482511520386, 0.07207386940717697, 0.05762749910354614, .
// ]
```

Example: Perform automatic speech recognition with OpenAI whisper on WebGPU (demo)

```
import { pipeline } from "@huggingface/transformers";

// Create automatic speech recognition pipeline
const transcriber = await pipeline(
  "automatic-speech-recognition",
  "onnx-community/whisper-tiny.en",
  { device: "webgpu" },
);
```

```
// Transcribe audio from a URL
const url = "https://huggingface.co/datasets/Xenova/transformers.js-do
const output = await transcriber(url);
console.log(output);
// { text: ' And so my fellow Americans ask not what your country can
```

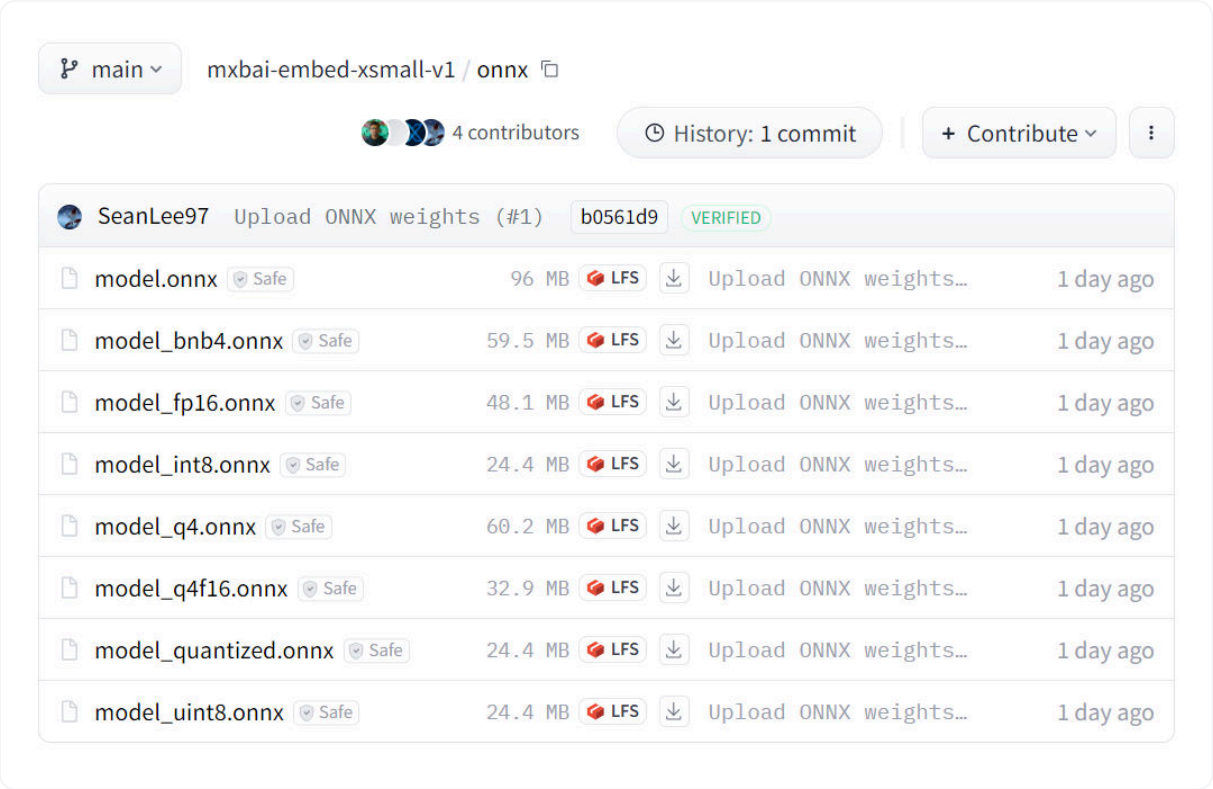
Example: Perform image classification with MobileNetV4 on WebGPU (demo)

```
import { pipeline } from "@huggingface/transformers";

// Create image classification pipeline
const classifier = await pipeline(
  "image-classification",
  "onnx-community/mobilenetv4_conv_small.e2400_r224_in1k",
  { device: "webgpu" },
);

// Classify an image from a URL
const url = "https://huggingface.co/datasets/Xenova/transformers.js-do
const output = await classifier(url);
console.log(output);
// [
//   { label: 'tiger, Panthera tigris', score: 0.6149784922599792 },
//   { label: 'tiger cat', score: 0.30281734466552734 },
//   { label: 'tabby, tabby cat', score: 0.0019135422771796584 },
//   { label: 'lynx, catamount', score: 0.0012161266058683395 },
//   { label: 'Egyptian cat', score: 0.0011465961579233408 }
// ]
```

New quantization formats (dtypes)

Before Transformers.js v3, we used the `quantized` option to specify whether to use a quantized (q8) or full-precision (fp32) variant of the model by setting `quantized` to `true` or `false`, respectively. Now, we've added the ability to select from a much larger list with the `dtype` parameter.

The list of available quantizations depends on the model, but some common ones are: full-precision (`"fp32"`), half-precision (`"fp16"`), 8-bit (`"q8"`, `"int8"`, `"uint8"`), and 4-bit (`"q4"`, `"bnb4"`, `"q4f16"`).



(e.g., mixedbread-ai/mxbai-embed-xsmall-v1)

Basic usage

Example: Run Qwen2.5-0.5B-Instruct in 4-bit quantization (demo)

```
import { pipeline } from "@huggingface/transformers";

// Create a text generation pipeline
const generator = await pipeline(
  "text-generation",
  "onnx-community/Qwen2.5-0.5B-Instruct",
  { dtype: "q4", device: "webgpu" },
);

// Define the list of messages
const messages = [
```

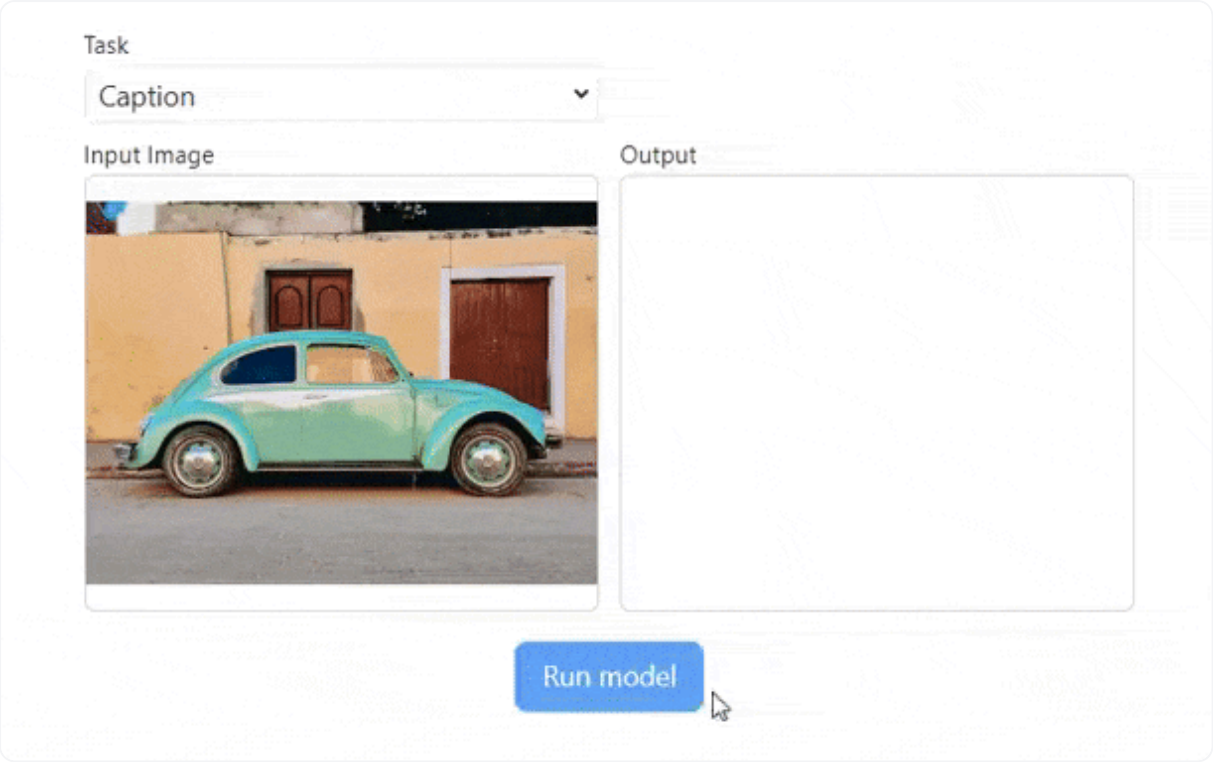
```
  { role: "system", content: "You are a helpful assistant." },
  { role: "user", content: "Tell me a funny joke." },
];

// Generate a response
const output = await generator(messages, { max_new_tokens: 128 });
console.log(output[0].generated_text.at(-1).content);
```

Per-module dtypes

Some encoder-decoder models, like Whisper or Florence-2, are extremely sensitive to quantization settings: especially of the encoder. For this reason, we added the ability to select per-module dtypes, which can be done by providing a mapping from module name to dtype.

Example: Run Florence-2 on WebGPU (demo)

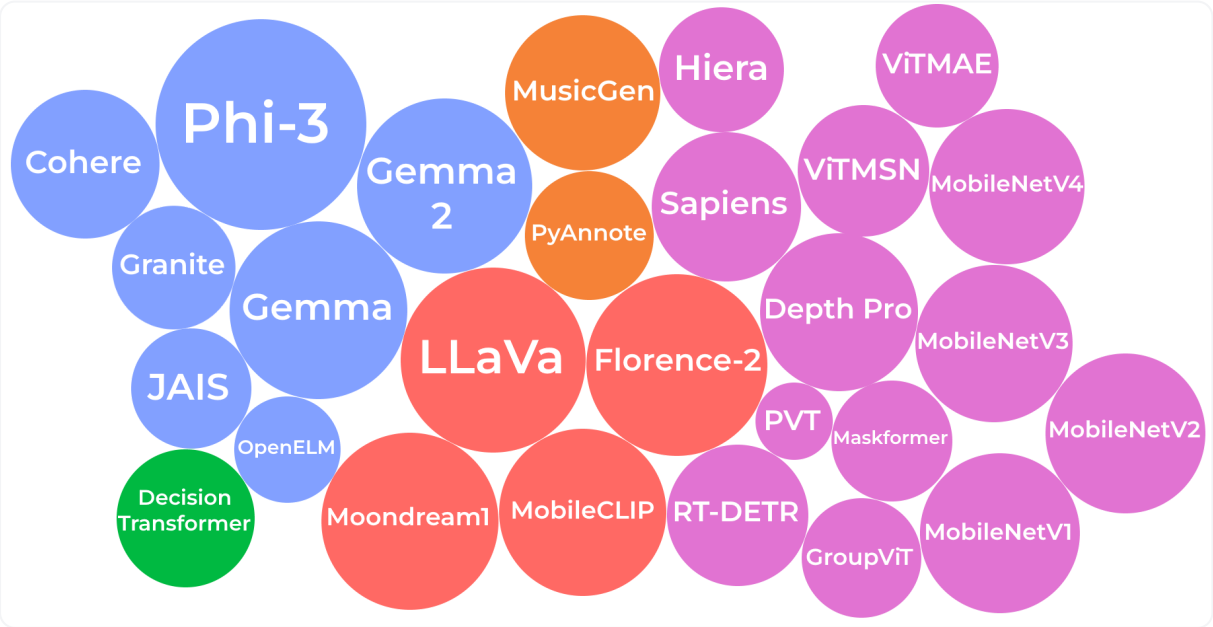
```
import { Florence2ForConditionalGeneration } from "@huggingface/transf

const model = await Florence2ForConditionalGeneration.from_pretrained(
  "onnx-community/Florence-2-base-ft",
  {
    dtype: {
      embed_tokens: "fp16",
      vision_encoder: "fp16",
      encoder_model: "q4",
      decoder_model_merged: "q4",
    },
    device: "webgpu",
  },
);
```



▶ See full code example

120 supported architectures

This release increases the total number of supported architectures to 120 (see full list), spanning a wide range of input modalities and tasks. Notable new names include: Phi-3, Gemma & Gemma 2, LLaVa, Moondream, Florence-2, MusicGen, Sapiens, Depth Pro, PyAnnote, and RT-DETR.

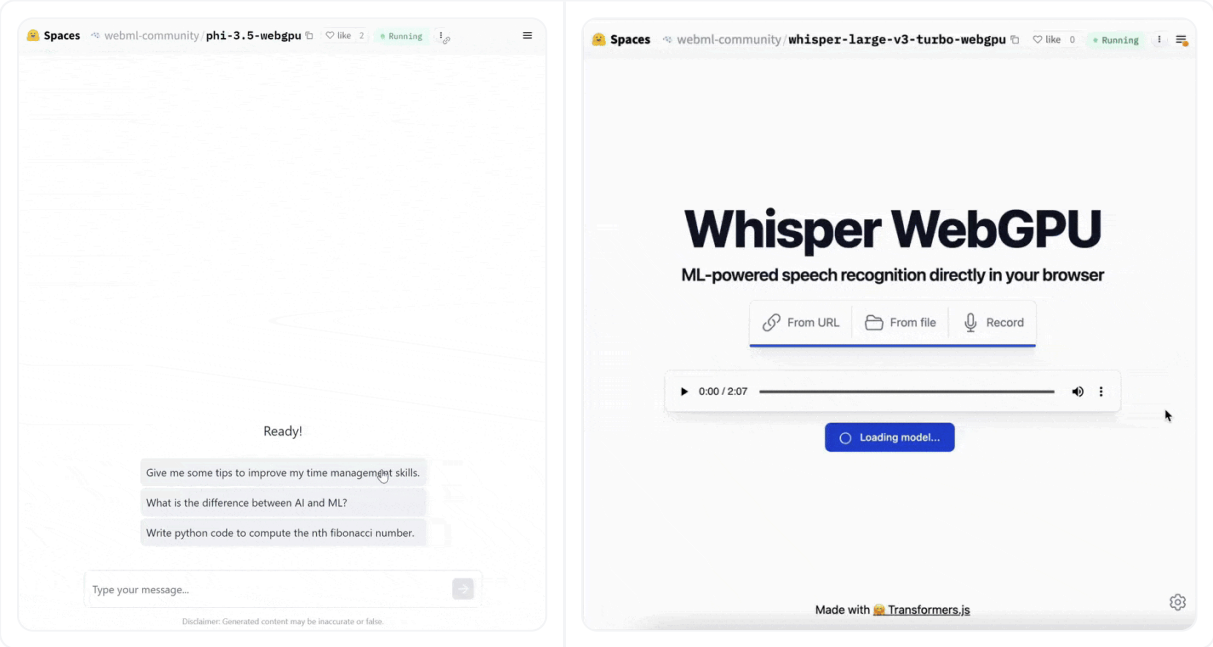


▶ List of new models

Example projects and templates

As part of the release, we've published 25 new example projects and templates, primarily focused on showcasing WebGPU support! This includes demos like Phi-3.5 WebGPU and Whisper WebGPU, as shown below.

> We're in the process of moving all our example projects and demos to https://github.com/huggingface/transformers.js-examples, so stay tuned for updates on this!



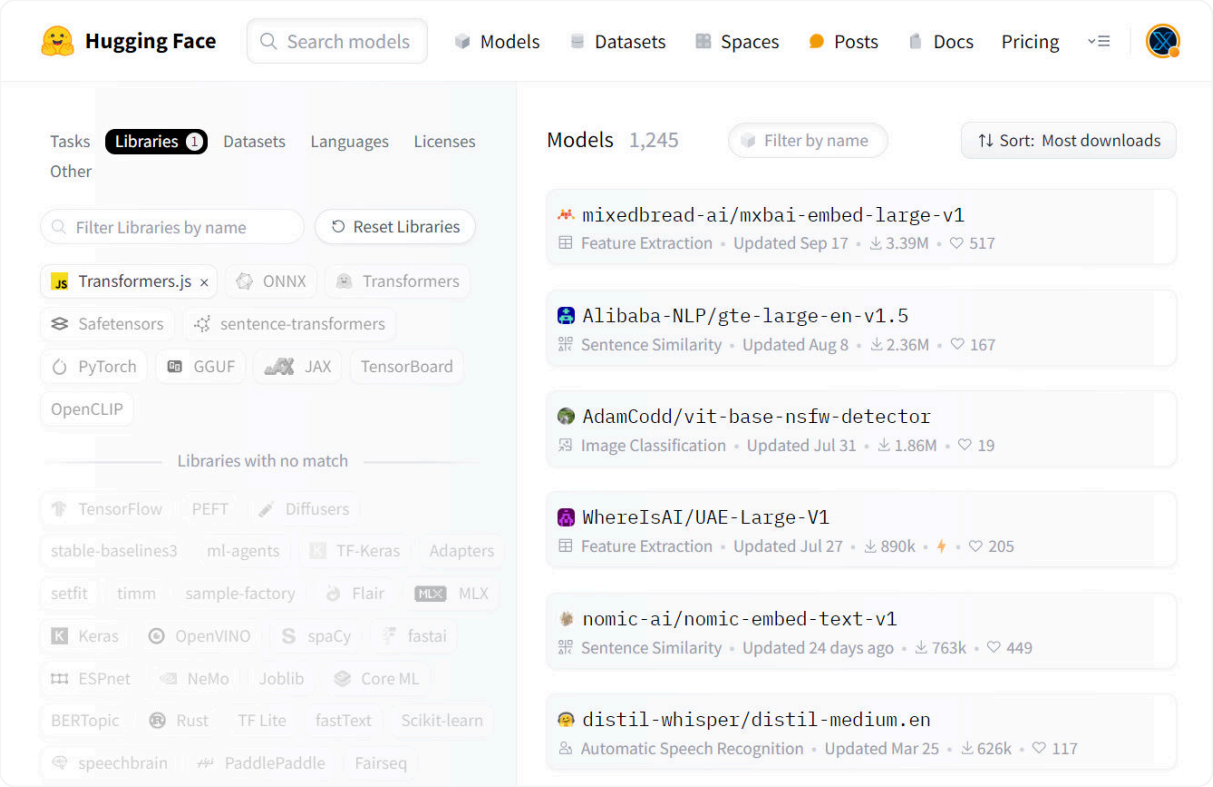
Over 1200 pre-converted models

As of today's release, the community has converted over 1200 models to be compatible with Transformers.js! You can find the full list of available models here.

If you'd like to convert your own models or fine-tunes, you can use our conversion script as follows:

```
python -m scripts.convert --quantize --model_id <model_name_or_path>
```

After uploading the generated files to the Hugging Face Hub, remember to add the `transformers.js` tag so others can easily find and use your model!



Node.js (ESM + CJS), Deno, and Bun compatibility

Transformers.js v3 is now compatible with the three most popular server-side JavaScript runtimes:

Runtime	Description	Examples
Node.js	A widely-used JavaScript runtime built on Chrome's V8. It has a large ecosystem and supports a wide range of libraries and frameworks.	ESM Example / CJS Example
Deno	A modern runtime for JavaScript and TypeScript that is secure by default. It uses ES modules and even features experimental WebGPU support.	Deno Example

Runtime	Description	Examples
Bun	A fast JavaScript runtime optimized for performance. It features a built-in bundler, transpiler, and package manager.	Bun Example

A new home on NPM and GitHub

Finally, we're delighted to announce that Transformers.js will now be published under the official Hugging Face organization on NPM as `@huggingface/transformers` (instead of `@xenova/transformers`, which was used for v1 and v2).

We've also moved the repository to the official Hugging Face organization on GitHub (https://github.com/huggingface/transformers.js), which will be our new home — come say hi! We look forward to hearing your feedback, responding to your issues, and reviewing your PRs!

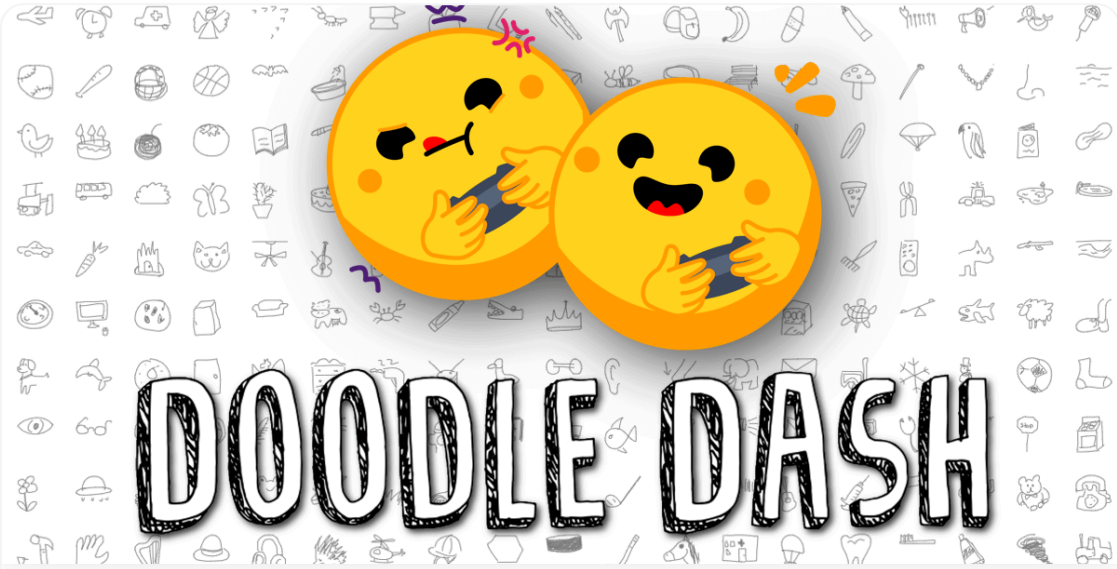
This is a significant milestone and we're extremely grateful to the community for helping us achieve this long-term goal! None of this would be possible without all of you… thank you! 🤗

More Articles from our Blog

Community Announcement Transformers

The Transformers Library: standardizing model definitions

△ 119 May 15, 2025



Game-Dev Guide Web

Making ML-powered web games with Transformers.js

Xenova △ 14 July 5, 2023

👏 Community

Edit Preview

Start discussing this article

Tap or paste here to upload images

Comment

Sign up or log in to comment

System theme

Company

TOS

Privacy

About

Jobs

Website

Models

Datasets

Spaces

Pricing

Docs